

```

import time
import tracemalloc

# Generate all possible moves
def generate_moves(state):
    moves = []

    # Convert tuple to list
    puzzle = list(state)

    # Find blank space (0)
    blank = puzzle.index(0)

    row = blank // 3
    col = blank % 3

    # UP
    if row > 0:
        new_puzzle = puzzle.copy()
        new_puzzle[blank], new_puzzle[blank - 3] = \
            new_puzzle[blank - 3], new_puzzle[blank]
        moves.append(tuple(new_puzzle))

    # DOWN
    if row < 2:
        new_puzzle = puzzle.copy()
        new_puzzle[blank], new_puzzle[blank + 3] = \
            new_puzzle[blank + 3], new_puzzle[blank]
        moves.append(tuple(new_puzzle))

    # LEFT
    if col > 0:
        new_puzzle = puzzle.copy()
        new_puzzle[blank], new_puzzle[blank - 1] = \
            new_puzzle[blank - 1], new_puzzle[blank]
        moves.append(tuple(new_puzzle))

```

```

# RIGHT
if col < 2:
    new_puzzle = puzzle.copy()
    new_puzzle[blank], new_puzzle[blank + 1] = \
        new_puzzle[blank + 1], new_puzzle[blank]
    moves.append(tuple(new_puzzle))

return moves

# DFS
def DFS(start_state, goal_state):

    # Create an empty STACK
    stack = []

    # Create an empty SET called VISITED
    visited = set()

    # PUSH start_state into STACK
    stack.append(start_state)

    while stack:

        # current_state = POP(STACK)
        current_state = stack.pop()

        # if current_state == goal_state
        if current_state == goal_state:
            return "Goal Found", len(visited)

        # if current_state not in VISITED
        if current_state not in visited:

            # Add current state to VISITED
            visited.add(current_state)

            # Generate all possible moves
            next_states = generate_moves(current_state)

```

```

        # Push unvisited states into stack
        for next_state in next_states:
            if next_state not in visited:
                stack.append(next_state)

    return "Goal Not Found", len(visited)

# Function to take 3x3 matrix input
def input_matrix(name):

    print(f"\nEnter {name} (3 x 3 matrix):")

    matrix = []

    for i in range(3):
        row = list(map(int, input(f"Enter row {i + 1}: ").split()))

        if len(row) != 3:
            print("Error: Enter exactly 3 numbers in each row.")
            exit()

        matrix.append(row)

    # Convert 3x3 matrix into a tuple
    state = tuple(num for row in matrix for num in row)

    return state

# Function to display matrix
def display_matrix(state):

    for i in range(0, 9, 3):
        print(state[i:i + 3])

# ----- MAIN PROGRAM -----

print("Enter 0 for the blank space.")

```

```
# Take initial state as 3x3 matrix
start = input_matrix("Initial State")

# Take goal state as 3x3 matrix
goal = input_matrix("Goal State")

# Validate input
if set(start) != set(range(9)) or set(goal) != set(range(9)):
    print("\nError: Matrix must contain numbers 0 to 8 exactly once.")
    exit()

# Display states
print("\nInitial State:")
display_matrix(start)

print("\nGoal State:")
display_matrix(goal)

# Start time and memory measurement
tracemalloc.start()

start_time = time.perf_counter()

# Run DFS
result, visited_count = DFS(start, goal)

end_time = time.perf_counter()

# Get memory usage
current_memory, peak_memory = tracemalloc.get_traced_memory()

tracemalloc.stop()

# Display result
```

```

print("Result:", result)
print("States Visited:", visited_count)

print("\nTime Complexity Measurement:")
print("Execution Time: {:.6f} seconds".format(
    end_time - start_time
))

print("\nSpace Complexity Measurement:")
print("Peak Memory Used: {:.2f} KB".format(
    peak_memory / 1024
))

print("\nTheoretical Complexity:")
print("Time Complexity : O(b^d)")
print("Space Complexity : O(b^d)")
print("where b = branching factor and d = search depth")

```

